Design and Analysis of Algorithm (DAA), Lab-10

BTech (CS-B and CE), 3rd Semester
October 13, 2026



Instructor: Dr. Ajaya Kumar Dash.

Questions based on graphs and their representations, disjoint set data structure/ Union-Find data structure/Disjoint Set Union (DSU), and connected components of a graph (Difficulty level: medium and hard)

1. **[Online Graph-Bipartiteness Testing]** A graph is bipartite if and only if it contains no odd-length cycles. Given an undirected graph with V vertices, edges arrive online one by one. Maintain whether the graph remains bipartite. If adding an edge (u, v) does not introduce an odd cycle, add it; otherwise, report that it violates bipartiteness and skip it. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

2. **[Dynamic Connected Components in a Grid]** Given an $R \times C$ grid initially completely filled with water, land cells are added one by one at positions $(r_1, c_1), (r_2, c_2), \ldots, (r_k, c_k)$. After each operation, output the current number of 4-directionally connected land components (islands). By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

   [*Hint*: This is the Dynamic Connected Components in a Grid (commonly known as Number of Islands II), which can be solved using a Disjoint Set Union (DSU / Union-Find) data structure with coordinate flattening.]

3. **[Maximum Weight Component with Size Constraints]** Given a graph where each vertex v has weight w_v, edges are added sequentially with weights. At any point, compute the maximum total vertex weight of any connected component whose size does not exceed a threshold K. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

4. **[Reverse Time-Step DSU Application]** Given a vertex-weighted undirected graph, vertices are sequentially deleted one by one in an order $P = [p_1, p_2, \ldots, p_V]$. After each vertex deletion, compute the maximum sum of vertex values among all remaining connected components. Note that deleting a vertex in a graph means removing that specific vertex and all edges connected (incident) to it from the graph structure. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

5. **[Lexicographically Smallest Connected Component Permutation]** Given a permutation A of numbers $1 \ldots n$, and a list of allowable pairwise swaps (u, v) forming edges in an undirected graph, find the lexicographically smallest permutation achievable through any number of valid swaps. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.